

Columbia College Computer Science

CCCS High school Programming Contest

Practice questions

CCCS High School Programming Contest website

https://yliow.github.io/programming-contest

March 6, 2027

We hope to see you there!!

Contents

1 Overview 3

2 Beginner 4

3 Intermediate 26

4 Advanced 37

5 Competitive Programming 56

1 Overview

The following is an assortment of practice questions that range from **Beginner** to **Advanced**. Note that the goal of this pdf is not to teach you programming – that would require a whole book.

The questions are not ordered in their respective categories, some may be harder than others and appear sooner or they may be easier and appear later. Beyond the given examples, they are left without specific test cases.

If you find a question doesn't look/sound right, you want clarification on something, or if you just want to talk about computer science feel free to message us on the CCCS High School Programming Contest Discord (`https://discord.gg/gGbqfrSeMD`).

Over the coming months before the programming contest (*March 6th, 2027*) we will be posting solutions and walkthroughs for some of the questions in the document.

It takes times to be a strong programmer or software engineer. Be patient. Good luck and have fun!

2 Beginner

1. Given an integer with at most 5 digits, print the digits.

```
Example 1:
Input: 13524
Output: 1 3 5 2 4

Example 2:
Input: 42
Output: 0 0 0 4 2
```

(If `x` is an integer variable, print `x % 10` and `x / 10`. Then print `x % 100` and `x / 100`. Then print `x % 1000` and `x / 1000`.)

2. Given an integer x print the number of digits of x.

```
Example 1:
Input: 13524
Output: 5

Example 2:
Input: 8
Output: 1
```

3. Given an integer, print 1 if the integer is a palindrome. Otherwise print 0. An integer is a palindrome if it's the same if read backwards. 1, 134431 and 13531 are palindromes. 135731 is not a palindrome.

```
Example 1:
Input: 134431
Output: 1

Example 2:
Input: 135731
Output: 0
```

4. Given two times in HHMMSS format, print the time between them as HH:MM:SS. Assume both times are on the same day and the second is not earlier than the first.

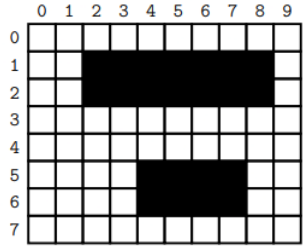
```
Example 1:
Input: 091400 091415
Output: 00:00:15

Example 2:
Input: 095045 110510
Output: 01:14:25
```

5. Given a month, year, and weekday of the first day, print that month's calendar. Weekdays are numbered 0 (Sunday) through 6 (Saturday). Account for leap years.

```
Example 1:
Input: month = 3, year = 2008, firstWeekday = 6
Output:
March 2008
Su  Mo  Tu  We  Th  Fr  Sa
                        1
 2   3   4   5   6   7   8
 9  10  11  12  13  14  15
16  17  18  19  20  21  22
23  24  25  26  27  28  29
30  31
```

6. Given a row r and column c, print 1 if the position is open and 0 if it is blocked or outside the map. Rows are numbered 0–7 and columns 0–9. The blocked areas are: Rows 1–2, columns 2–8. Rows 5–6, columns 4–7.




```
Example 1:
Input: r = 0, c = 0
Output: 1

Example 2:
Input: r = 1, c = 4
Output: 0
```

7. Given a positive integer n, print a V made of stars with n rows.

```
Example 1:
Input: 1
Output:
*

Example 2:
Input: 2
Output:
* *
 *

Example 3:
Input: 4
Output:
*   *
 * *
  *
```

8. Print all 3-by-3 magic squares, i.e., arrangements of the digits 1–9 where every row, column, and both diagonals add to 15. Use each digit exactly once. There is no input. Your program should find all 8 arrangements. Rotations and reflections count as different arrangements. The first two magic squares are shown below:

```
Example 1:
2 7 6
9 5 1
4 3 8

4 9 2
3 5 7
8 1 6
```

9. Compute and store the first 100,000 primes in an array. Print the first 5 and last 5. Then repeatedly read a number x and an option: Option 0: Use binary search to check whether x is prime. If it is, print its array index, starting at 0. Option 1: Print x as a product of prime factors. Stop when x is -1.
Assume x is at least 2 and any prime needed is in your array. For option 0, x will not exceed the largest stored prime.

```
Example 1:
Input: x = 13, option = 0
Output: Prime, index 5

Example 2:
Input: x = 12, option = 0
Output: Not prime

Example 3:
Input x = 60, option = 1
Output: 2^2 * 3 * 5
```

10. Given a positive integer n, print all positive integers x, y, and z where $x^3+y^3 = z$ and z is at most n. Print each solution as $x\ y\ z$, ordered by z and then x. Include reversed pairs.

```
Example 1:
Input: 16
Output:
1 1 2
1 2 9
2 1 9
2 2 16

Explanation: 1 2 9 means 1^3 + 2^3 = 9.
```

11. Given four integers a, b, c, and d, where a/b and c/d are viewed as fractions, print the sum of these two fractions. Simplify the result. If it is a whole number, print it without a denominator.

```
Example 1:
Input: 1 6 1 3
Output: 1/2

Example 2:
Input: 3 4 5 4
Output: 2

Example 3:
Input: 1 2 1 -2
Output: 0
```

12. Given a nonnegative integer, repeatedly add its digits until only one digit remains.

```
Example 1:
Input: 38
Output: 2
Explanation: 3 + 8 = 11, then 1 + 1 = 2.

Example 2:
Input: 0
Output: 0
```

13. Given an array representing the digits of a nonnegative integer, add 1 and return the resulting digits. Work directly with the array rather than converting the entire number to an integer.

```
Example 1:
Input: [1, 2, 3]
Output: [1, 2, 4]

Example 2:
Input: [9, 9]
Output: [1, 0, 0]
```

14. Given two sorted arrays, merge them into `nums1` in sorted order. The first m elements of `nums1` are actual values; its remaining `n` spaces are placeholders. `nums2` contains `n` actual values.

```
Example 1:
Input: nums1 = [1, 2, 3, 0, 0, 0]
       nums2 = [2, 5, 6]
       n = 3
       m = 3
Output: nums1 = [1, 2, 2, 3, 5, 6]

The placeholder zeros are ignored. Keep duplicate values.
```

15. Given a sorted array of distinct integers and a target, return the target's index. If it is missing, return the index where it would be inserted. Indices start at 0. Use binary search.

```
Example 1:
Input: [1, 3, 5, 6], target = 5
Output: 2

Example 2:
Input: [1, 3, 5, 6], target = 2
Output: 1

Example 3:
Input: [1, 3, 5, 6], target = 7
Output: 4
```

16. Given a string and a number of rows, place characters into rows by moving down, then up, repeatedly. Return the characters read one row at a time from top to bottom. For 3 rows, characters go into rows 0, 1, 2, 1, 0, 1, 2, and so on. If there is only one row, return the original string.

```
Example1:
Input: "PAYPALISHIRING", rows = 3
Output: "PAHNAPLSIIGYIR"
Explanation:
Row 0: PAHN
Row 1: APLSIIG
Row 2: YIR
```

17. Given an integer, return true if it is an ugly number, i.e., if it is positive and its only prime factors are 2, 3, and 5. Otherwise, return false. The number 1 also counts as ugly.

```
Example 1:
Input: 6
Output: true
Explanation: 6 = 2 * 3.

Example 2:
Input: 14
Output: false
Explanation: 14 has a prime factor of 7.
```

18. Given an integer array and a target, return the indices of two elements that add to the target. Indices start at 0. There is exactly one valid pair, and you cannot use the same element twice.

```
Example 1:
Input: [2, 7, 11, 15], target = 9
Output: [0, 1]

Example 2:
Input: [3, 3], target = 6
Output: [0, 1]
```

19. Given a nonnegative integer x, return its square root rounded down. Do not use built-in square-root or exponent functions.

```
Example 1:
Input: 4
Output: 2

Example 2:
Input: 8
Output: 2
Explanation: 2 * 2 is at most 8, but 3 * 3 is greater than 8.
```

20. Given an array of strings, return the longest beginning shared by every string. Return "" if there is no shared beginning.

```
Example 1:
Input: ["flower", "flow", "flight"]
Output: "fl"

Example 2:
Input: ["dog", "racecar", "car"]
Output: ""
```

21. Given a positive integer n, count the numbers from 1 through n that become different valid numbers when every digit is replaced using these rules:
    - 0, 1, and 8 stay unchanged.
    - 2 becomes 5, and 5 becomes 2.
    - 6 becomes 9, and 9 becomes 6.
    - 3, 4, and 7 make the number invalid.
    - Keep the digits in the same positions. Count a number only if all digits are valid and at least one changes.

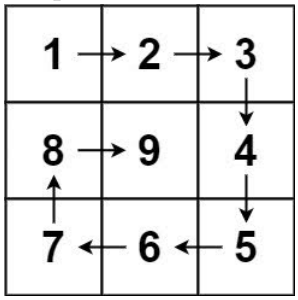
```
Example 1:
Input: 10
Output: 4
Explanation: The qualifying numbers are 2, 5, 6, and 9.
```

3 Intermediate

1. Given a string, swap the vowels on opposite sides of the string.

```
Example:
Input: "IceCreAm"
Output: "AceCreIm"
```

2. You are given a positive integer n that specifies the size of an $n \times n$ matrix. Your goal is to construct a matrix of that size where the elements are ordered in spiral order between 1 and n^2 (image below)

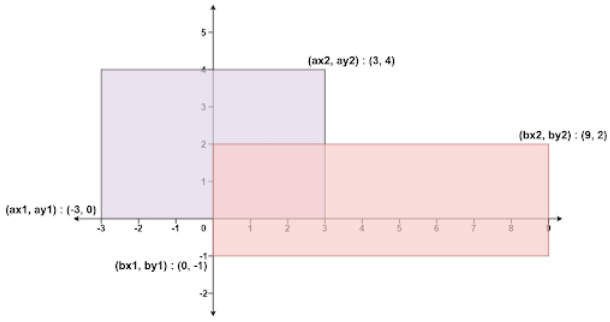



```
Example 1:
Input: 3
Output: [[1,2,3],[8,9,4],[7,6,5]]

Example 2:
Input: 2
Output: [[1,2],[4,3]]

Example 3:
Input: 1
Output: [[1]]
```

3. Given the coordinates of the corners for 2 rectangles, find the area of the 2 rectangles.




```
Example 1:
Input: -3, 0, 3, 4, 0, -1, 9, 2
Output: 45
Explanation:
6*4 = 24 and 9*3 = 27 would be 51, but they overlap.
The overlapping area is 3*2=6, so the total area of the rectangles is 45.
```

4. Given an array of integers in the range `[0,n]`, find the missing number in that range.

```
Example 1:
Input: [3,0,1]
Output: 2

Example 2:
Input: [9,6,4,2,3,5,7,0,1]
Output: 8
```

5. Given an integer n, return the count of prime numbers less than n.

```
Example:
Input: 10
Output: 4
Explanation: 2, 3, 5, 7
```

6. Given a string representing an Excel column title, find the corresponding column number.

```
Example 1:
Input: "A"
Output: 1

Example 2:
Input: "AA"
Output: 27

Example 3:
Input: "ZY"
Output: 701
```

7. Given an integer, convert it to a string representing an Excel column title.

```
Example 1:
Input: 5
Output: "E"

Example 2:
Input: 27
Output: "AA"

Example 3:
Input: 701
Output: "ZY"
```

8. Given an array of integers representing the maximum number of indexes you can move, check if you can reach the end of the array.
   You start at index 0, you want to reach the end, you can move UP TO n spaces from your current position where n is the value at arr[index]; return true or false depending on if you can or cannot reach the end

```
Example 1:
Input: [2, 3, 1, 1, 4]
Output: true

Example 2:
Input: [3, 2, 1, 0, 4]
Output: false
```

9. You are given an array of intervals, each element of this array is another array with length 2 that specified the start and end of the interval as follows: [start, end]. Your goal is to merge any overlapping intervals.

```
Example 1:
Input: [[1,2],[2,3],[3,5],[8,18]]
Output: [[1,3],[3,5],[8,18]]
Explanation: Since intervals [1,2] and [2,3] overlap, merge them into
    [1,3].

Example 2:
Input: [[1,5],[4,9]]
Output: [[1,9]]
Explanation: Intervals [1,5] and [4,9] are considered overlapping.

Example 3:
Input: [[0, 10],[1,2], [3,4], [4,5]]
Output: [[0, 10]]
Explanation: All intervals overlap with [0,10]
```

10. Given a string consisting of words separated by spaces, modify that string according to Goat Latin rules.
    - If a word starts with a vowel, add "ma" to the end of that word
      "`apple`" becomes "`applema`"
    - If a word starts with a consonant, move the first letter to the end of the word and add "ma"
      "`goat`" becomes "`oatgma`"
    - Add n "a"'s to the end of each word, where n is the word's index + 1

```
Example:
Input: "goats eat apples"
Output: "oatsgmaa eatmaaa applesmaaaa"
```

4 Advanced

1. Given an array of integers, can you rotate them k steps to the right?

```
Example 1:
Input: [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]

Example 2:
Input: [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

2. Given a signed 32-bit integer x, return x with its digits reversed.
   If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31-1}]$, then return 0.
   64-bit integers (signed or unsigned) are not allowed.

```
Example 1:
Input 123
Output: 321

Example 2:
Input: -123
Output: -321

Example 3:
Input: 120
Output: 21
```

3. Given two non-negative integers num1 and num2 represented as strings, return the product of num1 and num2, also represented as a string.
   Note: You must not use any built-in BigInteger library or convert the inputs to integers directly.

```
Example 1:
Input: num1 = "2", num2 = "3"
Output: "6"

Example 2:
Input: num1 = "123", num2 = "456"
Output: "56088"
```

4. Given a string of words, reverse their order of appearance. Remove all leading / trailing whitespace

```
Example:
Input: "the sky is blue"
Output: "blue is sky the"

Example:
Input: "  hello world  "
Output: "world hello"
```

5. Given a list of numbers, determine exactly how many numbers are strictly smaller than the current number for every single element in the list. Count all the strictly smaller values across the entire list and return these counts in a new list that corresponds to the exact order of the original numbers.

```
Example 1:
Input: [8,1,2,2,3]
Output: [4,0,1,1,3]
```

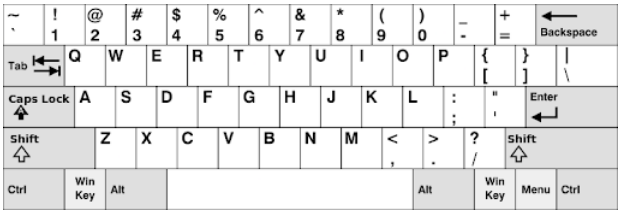
6. Imagine a black-and-white image represented as a 2D grid of 0s and 1s. You need to apply two transformations to this grid in order. First, flip the image horizontally, meaning the elements of each row are reversed. Then, invert the image, meaning every 0 becomes a 1 and every 1 becomes a 0. Return the final transformed grid.

```
Example 1:
Input: [[1,1,0],[1,0,1],[0,0,0]]
Output: [[1,0,0],[0,1,0],[1,1,1]]
```

7. You are given a main text string and a target substring. You need to continuously search for the leftmost appearance of the target substring within the main text and delete it. After deleting a substring, the remaining pieces of the main text will slide together, which might form a brand new occurrence of the target. Continue this deletion process until the target substring can no longer be found anywhere, and return the final string.

```
Example 1:
Input: "daabcbaabcbc", target = "abc"
Output: "dab"
```

8. You are given a list of words. Filter this list and return only the words that can be typed using just a single row of a standard English QWERTY keyboard. For reference, the top row is "qwertyuiop", the middle row is "asdfghjkl", and the bottom row is "zxcvbnm". Your logic should be completely unaffected by uppercase or lowercase letters.




```
Example 1:
Input: ["Hello", "Alaska", "Dad", "Peace"]
Output: ["Alaska", "Dad"]
```

9. Given an array of positive numbers and a target divisor, find the length of the shortest contiguous subarray that you can remove so that the sum of the remaining elements is perfectly divisible by the target divisor.
   If it is already divisible, you remove nothing. If it is impossible to achieve this, or if you would need to remove the entire array to make it work, consider it a failure and return -1.

```
Example 1:
Input: [3,1,4,2], divisor = 6
Output: [1]
```

10. You are given an integer to convert into its Roman numeral equivalent.
    Roman numerals are represented by seven different symbols: I (1), V (5), X (10), L (50), C (100), D (500), and M (1000). They are typically written from largest to smallest, left to right. However, there are subtractive exceptions for 4 (IV), 9 (IX), 40 (XL), 90 (XC), 400 (CD), and 900 (CM).

```
Example 1:
Input: 58
Output: "LVIII"
```

11. Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements. Note that you must do this in-place without making a copy of the array.

```
Example 1:
Input: [0,1,0,3,12]
Output: [1,3,12,0,0]
```

12. You are given a string that contains regular letters and star symbols (`'*'`). Whenever you encounter a star in the text, you must remove the star itself along with the closest non-star character immediately to its left. Process the text and return the final string after all stars have been applied and removed.

```
Example 1:
Input: "hello wo**rld"
Output: "hello rld"

Example 2:
Input: "what*s up"
Output: "whas up"
```

For this section, you should know your data structures and algorithms. If you don't, feel free to look anyways, but keep in mind these questions are very challenging even to those who do know their DSA.

1. Verify that a given number is valid. Given a string s, return whether s is a valid number.
   For example, all the following are valid numbers: “2”, “0089”, “-0.1”, “+3.14”, “4.”, “-.9”, “2e10”, “-90E3”, “3e+7”, “+6e-1”, “53.5e93”, “-123.456e789”, while the following are *not* valid numbers: “abc”, “1a”, “1e”, “e3”, “99e2.5”, “–6”, “-+3”, “95a54e53”.
   Formally, a valid number is defined using one of the following definitions:
   - An integer number followed by an optional exponent.
   - A decimal number followed by an optional exponent.
   - An integer number is defined with an optional sign ’-’ or ’+’ followed by digits.

   A decimal number is defined with an optional sign ‘-’ or ‘+’ followed by one of the following definitions:
   - Digits followed by a dot ‘.’.
   - Digits followed by a dot ‘.’ followed by digits.
   - A dot ‘.’ followed by digits.
   - An exponent is defined with an exponent notation ‘e’ or ‘E’ followed by an integer number.

   The digits are defined as one or more digits.

```
Example 1:
Input: "0"
Output: true

Example 2:
Input: "e"
Output: false

Example 3:
Input: "."
Output: false
```

2. Given an unsorted integer array, return the smallest positive integer that is not present. You must implement an algorithm that runs in $O(n)$ time and uses $O(1)$ auxiliary space.

```
Example 1:
Input: [1,2,0]
Output: 3
Explanation: The numbers in the range [1,2] are all in the array.

Example 2:
Input: [3,4,-1,1]
Output: 2
Explanation: 1 is in the array but 2 is missing.

Example 3:
Input: [7,8,9,11,12]
Output: 1
Explanation: The smallest positive integer 1 is missing.
```

3. You are given an integer array representing coins of different denominations and an integer amount representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin.

```
Example 1:
Input: [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1

Example 2:
Input: [2], amount = 3
Output: -1

Example 3:
Input: [1], amount = 0
Output: 0
```

4. Given an integer array and an integer k, return the kth largest element in the array. Note that it is the kth largest element in the sorted order, not the kth distinct element. Can you solve it without sorting?

```
Example 1:
Input: [3,2,1,5,6,4], k = 2
Output: 5

Example 2:
Input: [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

5. You are given a string s. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string `"ababcc"` can be partitioned into `["abab", "cc"]`, but partitions such as `["aba", "bcc"]` or `["ab", "ab", "cc"]` are invalid.
   Note that the partition is done so that after concatenating all the parts in order, the resultant string should be s.
   Return a list of integers representing the size of these parts.

```
Example 1:
Input: "ababcbacadefegdehijhklij"
Output: [9,7,8]
Explanation:
The partition is "ababcbaca", "defegde", "hijhklij".
This is a partition so that each letter appears in at most one part.
A partition like "ababcbacadefegde", "hijhklij" is incorrect, because it
    splits s into less parts.

Example 2:
Input: "eccbbbbdec"
Output: [10]
```

5 Competitive Programming

If you've used sites like LeetCode or HackerRank, you'll have noticed some of these problems are familiar.

For those of you not acquainted with these sites, they are places where you can find problems like the ones you may have solved in this document. There are a **lot** of different kinds of solutions to even the simplest problems.

Some companies actually take questions from these sites and use them in interviews, so not all of the aspects of competitive programming are just for the sake of programming.

Try out both the sample questions below to get used to the interface as you could end up using it a lot some day:

1. Two Sum - LeetCode
2. Solve Me First - HackerRank

After you're done with that, go explore! There are thousands of problems on these sites so you'll almost never run out of things to do.